MBLP131 · HAFTA 6

İşletim Sistemleri Temelleri

Hafta 6 · Bellek Yönetimi: Bitişik Tahsis, Sayfalama ve Bölütleme

Birçok program, tek bir RAM: her süreç kendine ait, korunmuş bir bellek parçasını nasıl alır?

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 5. Hafta

Zamanlama algoritmaları

FCFS, SJF/SRTF, Öncelik ve Round Robin; Gantt şemaları ve ortalama bekleme süresiyle karşılaştırıldı.

Zaman dilimi

Çok küçük dilimli Round Robin zamanı bağlam değişimlerine harcar; çok büyük dilim FCFS'ye dönüşür.

Yarış durumları

İş parçacıkları ortak veriyi eşgüdümsüz güncellerse sonuç zamanlamaya bağlı olur. Kritik bölge korunmalıdır.

Mutex ve semafor

lock()/unlock() ve wait()/signal() karşılıklı dışlama sağlar; iş parçacıkları işlemciyi boşa yakmadan bekler.

5. hafta işlemciyi zaman içinde paylaştırdı. Bugün ana belleği alan olarak paylaştırıyoruz: RAM'de aynı anda birçok süreç, her biri diğerlerinden korunmuş.

Bugünün Gündemi

1. **Temeller** Adres bağlama, mantıksal ve fiziksel adres, MMU, taban ve sınır yazmaçları, takas.
2. **Bitişik tahsis** Delikler, ilk uyan / en iyi uyan / en kötü uyan, parçalanma, sıkıştırma.
3. **Sayfalama** Sayfa ve çerçeve, sayfa tablosu, adres çevirisi, TLB, etkin erişim süresi.
4. **Bölütleme** Belleğe programcının gözüyle bakış, bölüt tablosu ve x86-64 Linux'un gerçekte yaptığı.
5. **Uygulama** Linux'ta gerçek adresler: /proc/self/maps, fork() ve iki kez aynı adres.

BÖLÜM 1

Bellek Yönetiminin Temelleri

Tek bir RAM'de birçok süreç ve hiçbiri bir başkasının belleğine dokunamaz.

MOTİVASYON

Bellek Neden Yönetilmeli?

İşlemci yalnızca ana bellekteki komutları yürütebilir ve verileri okuyabilir (yazmaçlar ve önbellek, RAM'in kopyalarıdır).

Aynı anda birçok süreç

- Çoklu programlama (1. hafta), işlemci boş kalmasın diye RAM'de birkaç süreç tutar.
- RAM'in hangi kısmı kime verilecek?

Koruma

- Bir süreç başka bir sürecin ya da çekirdeğin belleğini okuyamamalı, üzerine yazamamalı.
- Her erişimi yazılımla denetlemek çok yavaştır: bunu donanım yapmalıdır.

Yer değiştirme

- Program RAM'in neresine yükleneceğini önceden bilmez.
- Hatta çalışırken başka bir yere taşınabilir.

Hatırlatma: her sürecin düzeni metin · veri · öbek (heap) · yığıt (stack) şeklindedir (3. hafta). Bugünkü soru: bu baytlar RAM'de gerçekte nerede durur?

Bir Ad Ne Zaman Adrese Dönüşür?

Program değişkenlere adlarıyla başvurur; donanım ise sayılara ihtiyaç duyar. Bağlama üç farklı zamanda yapılabilir.



Derleme zamanı

Son konum biliniyorsa derleyici mutlak adresler üretebilir. Programı taşımak yeniden derlemek demektir. (MS-DOS .COM dosyaları)

Yükleme zamanı

Derleyici yer değiştirebilir kod üretir ("başlangıçtan 14 bayt ötede"); adresleri yükleyici düzeltir. Yüklendikten sonra program taşınamaz.

Yürütme zamanı

Adresler her erişimde donanım tarafından çevrilir; süreç çalışırken taşınabilir. Tüm modern işletim sistemleri bunu kullanır.

Mantıksal ve Fiziksel Adres

Mantıksal (sanal) adres

- Program çalışırken işlemcinin ürettiği adres.
- Programın (ve printf("%p") ile sizin) gördüğü adres.
- Her süreç 0 adresinden başlayabilir.
- **Mantıksal adres uzayı:** bir sürecin tüm mantıksal adreslerinin kümesi.
- Örnek: sonraki slayttaki MMU örneğinde 346; C programımızın yazdırdığı 0x55f2….

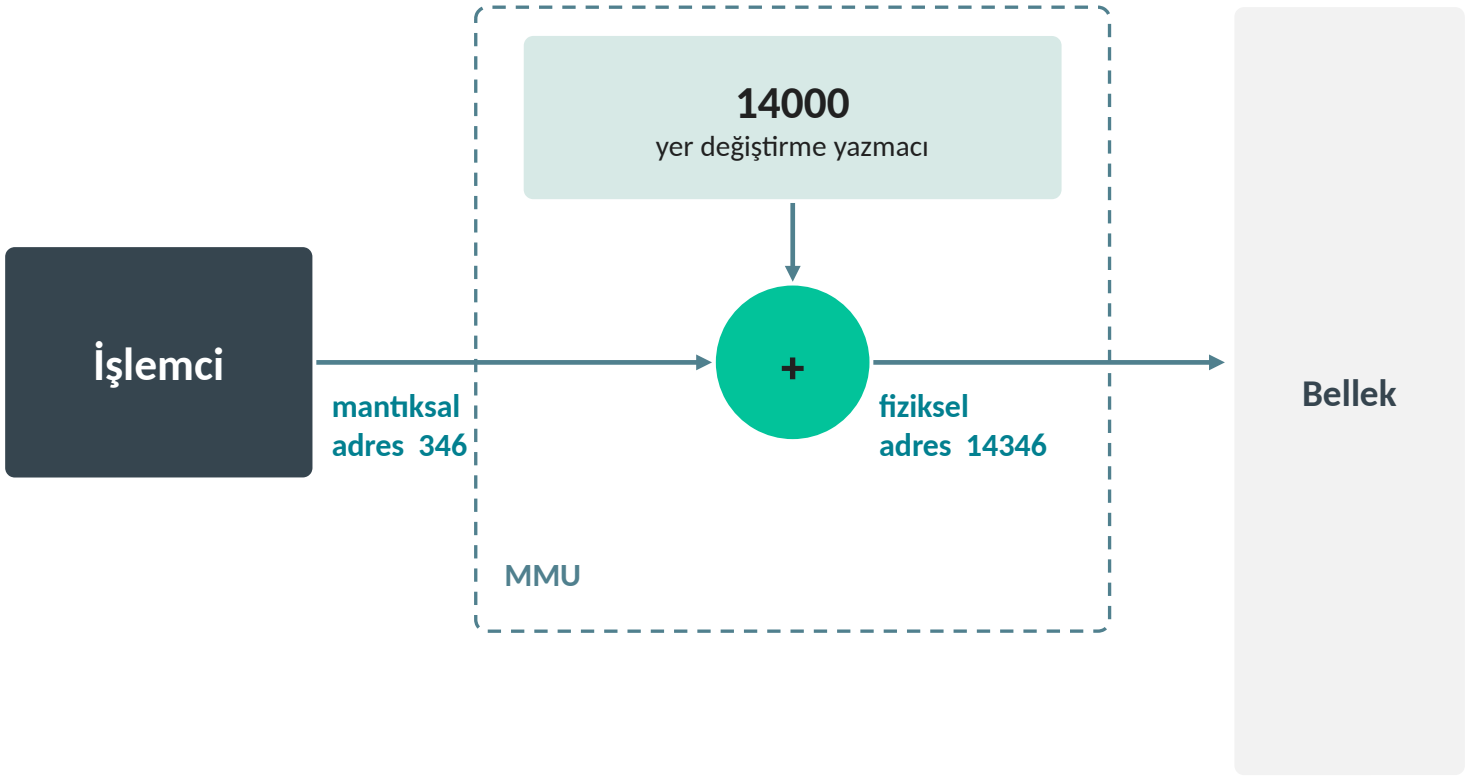
Fiziksel adres

- Bellek yoluna (bus) gerçekten konan adres.
- Yalnızca bellek donanımı görür.
- RAM'in her baytının tek bir fiziksel adresi vardır.
- **Fiziksel adres uzayı:** bunlara karşılık gelen fiziksel adreslerin kümesi.
- Örnek: 14346, yani mantıksal 346'nın RAM'de gerçekte vardığı yer.

Derleme ya da yükleme zamanında bağlamada ikisi aynıdır. Yürütme zamanında bağlamada farklıdır ve birini diğerine bir donanım birimi, bellek yönetim birimi (memory-management unit, MMU) çevirir.

Yer Değiştirme Yazmaçlı MMU

En basit MMU, işlemcinin ürettiği her adrese bir yer değiştirme (taban) yazmacının değerini ekler.

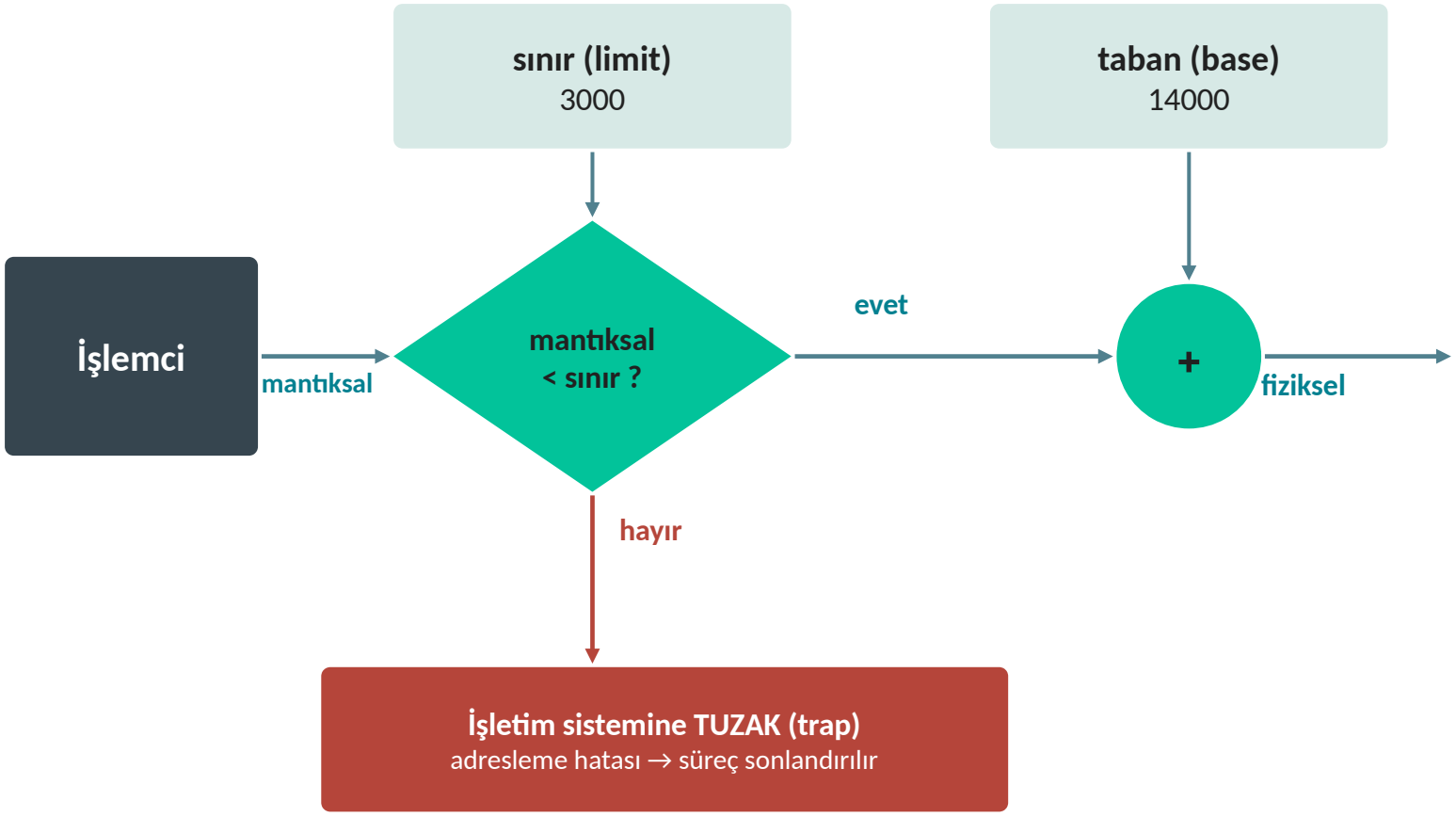


Bunun anlamı

- Program 0 … 2999 adreslerinde çalıştığını sanır.
- İşletim sistemi onu 14000'e yükledi ve yer değiştirme yazmacına 14000 yazdı.
- Süreci taşımak için işletim sistemi onu kopyalar ve tek bir yazmacı değiştirir.
- Bu yazmacı yüklemek ayrıcalıklı bir komuttur: yalnızca çekirdek yapabilir (2. hafta).

Taban ve Sınır Yazmaçları: Koruma Denetimi

İkinci bir yazmaç, sınır (limit), sürecin boyunu tutar. Her erişim, yer değiştirmeden önce denetlenir.



Kurallar

- Geçerli: 0 ≤ mantıksal < sınır.
- Fiziksel = taban + mantıksal.
- Bu yazmaçları yalnızca çekirdek yükleyebilir.
- Linux'ta tuzak "Segmentation fault" olarak görünür.

ÇÖZÜMLÜ ÖRNEK

Onluk ve Onaltılık Tabanda Taban ve Sınır

Taban	Sınır	Mantıksal adres	Denetim: mantıksal < sınır?	Sonuç
14000	3000	346	346 < 3000 ✓	fiziksel 14346
14000	3000	2999	2999 < 3000 ✓	fiziksel 16999 (son geçerli bayt)
14000	3000	3500	3500 < 3000 ×	TUZAK: adresleme hatası
0x4000	0x1000	0x02A4	0x02A4 < 0x1000 ✓	fiziksel 0x42A4 (= 17060)
0x4000	0x1000	0x1200	0x1200 < 0x1000 ×	TUZAK: adresleme hatası

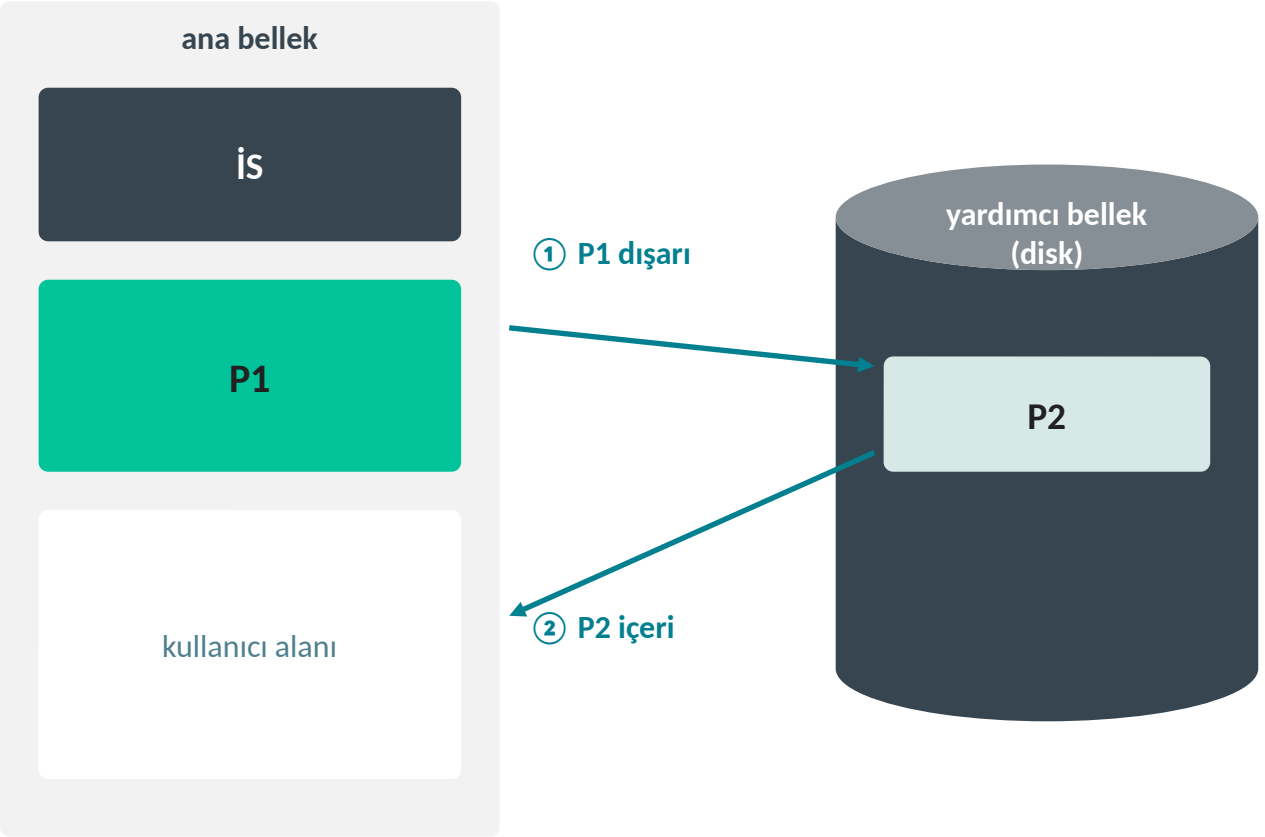
fiziksel bellek (onluk satır): süreç yalnızca nane yeşili aralığa dokunabilir



Süreç tam olarak [taban, taban + sınır) fiziksel aralığına sahiptir: onluk tabanda 14000 … 16999, onaltılıkta 0x4000 … 0x4FFF (0x4000 = 16384, 0x1000 = 4096).

Takas (Swapping): Bütün Süreci Diske Taşımak

RAM dolarsa işletim sistemi bir süreci yardımcı belleğe (backing store, disk) kopyalayıp daha sonra geri getirebilir.



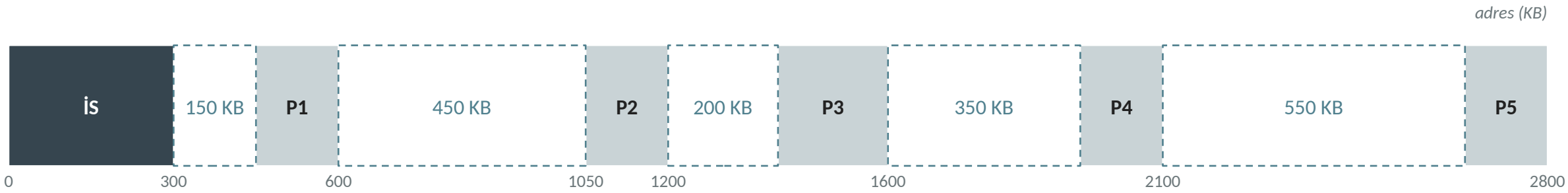

```
Süreç boyu        100 MB
Disk aktarımı      50 MB/s

dışarı: 100 / 50 = 2 s
içeri : 100 / 50 = 2 s
toplam ≈ 4 s   (+ disk gecikmesi)
```

4 s, 10 ms'lik 400 Round Robin dilimi demektir! Bu yüzden modern sistemler bütün süreçleri değil, tek tek sayfaları takas eder (50 MB/s'de 4 KB ≈ 80 µs), bkz. 7. hafta.

Değişken Bölümler ve Delik Listesi

Her süreç tam ihtiyacı kadar, bitişik tek bir blok alır. Süreçler arasındaki boş bloklara delik (hole) denir.



İşletim sisteminin tuttuğu delik listesi

- Her deliğin (başlangıç, boy) çifti, burada adres sırasıyla:
- (300, 150) · (600, 450) · (1200, 200) · (1600, 350) · (2100, 550)
- Toplam boş: 1700 KB, 5 parça hâlinde.

İşletim sistemi ne yapar?

- **Süreç gelince:** yeterince büyük bir delik bulur, gerekeni alır, kalanı daha küçük bir delik olarak bırakır.
- **Süreç bitince:** bloğu deliğe dönüşür; komşu deliklerle birleştirilir.

İlk Uyan, En İyi Uyan, En Kötü Uyan

Dört süreç bu sırayla geliyor: A = 220 KB, B = 400 KB, C = 130 KB, D = 430 KB. Her biri hangi deliği alır?

İlk uyan (first-fit)

- Yeterince büyük ilk deliği (en düşük adresli) al.
- Hızlı: arama erken durabilir.

En iyi uyan (best-fit)

- Yeterince büyük en küçük deliği al.
- En küçük artığı bırakır; tüm listeyi taramalı (ya da listeyi boya göre sıralı tutmalı).

En kötü uyan (worst-fit)

- En büyük deliği al.
- Fikir: artık işe yarayacak kadar büyük kalır. O da her şeyi taramalı.

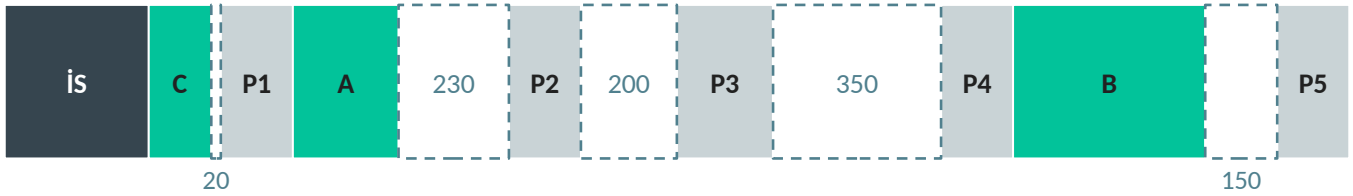
```
delikler (boy@başlangıç, KB):
150@300  450@600  200@1200  350@1600  550@2100

A = 220:  ilk → 450@600   en iyi → 350@1600
          en kötü → 550@2100
```

Benzetimler (Silberschatz): ilk uyan ve en iyi uyan, hem zaman hem bellek kullanımında en kötü uyandan iyidir; ilk uyan genellikle daha hızlıdır.

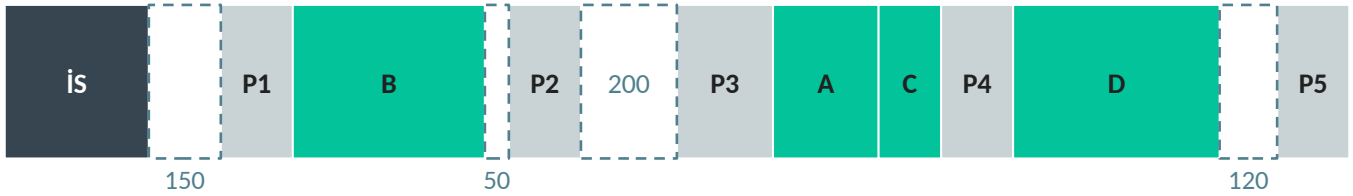
Aynı İstekler, Üç Farklı Sonuç

İlk uyan



D beklemek zorunda: 950 KB boş, ama ≥ 430 KB delik yok.

En iyi uyan



Dördü de yerleşti.

En kötü uyan



D beklemek zorunda: 950 KB boş, ama ≥ 430 KB delik yok.

yeni süreç mevcut süreç delik (boyu KB)

Küçük deliklerin (ör. 20 KB) etiketi çubuğun altında.

Dış ve İç Parçalanma

Dış parçalanma (external fragmentation)

- Toplamda yeterince boş bellek var, ama her biri tek başına küçük deliklere bölünmüş.
- Yukarıda ilk uyan: 950 KB boş, D'nin 430 KB'a ihtiyacı var, en büyük delik 350 KB.
- Değişken bölümlerin ve bölütlemenin tipik sorunu.
- **Çareler:** sıkıştırma (sonraki slayt) ya da sürecin bitişik olmamasına izin vermek (sayfalama, 2. bölüm).

İç parçalanma (internal fragmentation)

- Sürece verilmiş ama onun kullanmadığı bellek: bloğun içindeki israf.
- Bellek sabit boyutlu birimlerle dağıtılınca olur: 18.462 bayt istenir, 18.464 verilir.
- Sabit bölümlerin ve sayfalamanın tipik sorunu.
- **Çare:** daha küçük tahsis birimleri; bedeli daha fazla kayıt tutma (daha büyük tablolar).

%50 kuralı (Silberschatz): ilk uyanda, tahsis edilmiş N blok için yaklaşık 0,5 N blok daha parçalanmaya gider; belleğin üçte birine kadarı kullanılamaz hâle gelebilir.

Sıkıştırma (Compaction): Belleği Bir Araya Toplamak

Tüm süreçleri bir uca taşıyın; böylece bütün delikler tek bir büyük delikte birleşir.

Önce (ilk uyan sonucu)



Sıkıştırmadan sonra: 950 KB'lık tek delik → D (430 KB) sığar



- **Yalnızca** yürütme zamanında bağlamada mümkün: taşıdıktan sonra yer değiştirme yazmacı güncellenir.

- **Pahalı:** RAM'in tamamına kadarını kopyalar; süreçler durmalıdır ve taşınan bir tampona G/Ç yapılması sorun yaratır.

ARA

10 Dakika Mola

Sırada: dış parçalanmayı ortadan kaldıran fikir, sayfalama.

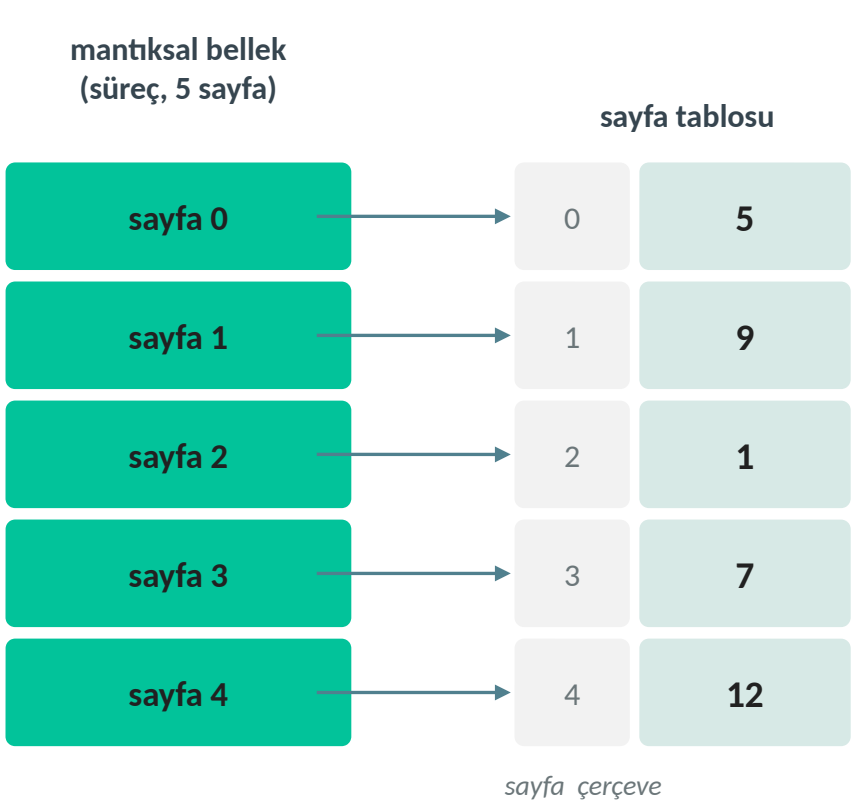
BÖLÜM 2

Sayfalama (Paging)

Belleği eşit parçalara bölün ve sürecin parçalarını RAM'in her yerine dağıtın.

Sayfalar, Çerçeveler ve Sayfa Tablosu

Mantıksal bellek sayfalara (page), fiziksel bellek aynı boyda çerçevelere (frame) bölünür (burada 4 KB). Her sayfa her boş çerçeveye konabilir.





Önemli noktalar

- Sayfa boyu = çerçeve boyu, 2'nin bir kuvveti (Linux/x86: 4 KB).
- Bir sürecin çerçeveleri bitişik ya da sıralı olmak zorunda değildir.
- Her boş çerçeve kullanılabilir → dış parçalanma yok.
- Her sürecin bir sayfa tablosu var; işletim sistemi bir boş çerçeve listesi tutar.

Adresi Sayfa Numarası ve Ofsete Bölmek

m bitlik mantıksal adreste ve 2^n baytlık sayfada, üstteki m − n bit sayfa numarası p, alttaki n bit ofset (sayfa içi konum) d'dir.




```
p = adres div 2^n   =  adres >> n
d = adres mod 2^n   =  adres & (2^n - 1)
fiziksel = çerçeve(p) × 2^n + d

16 bit adres, 4 KB sayfa (n = 12):  p = 4 bit, d = 12 bit
32 bit adres, 4 KB sayfa (n = 12):  p = 20 bit, d = 12 bit

0x3A7F = 0011 | 1010 0111 1111
         p = 3   d = 0xA7F
```

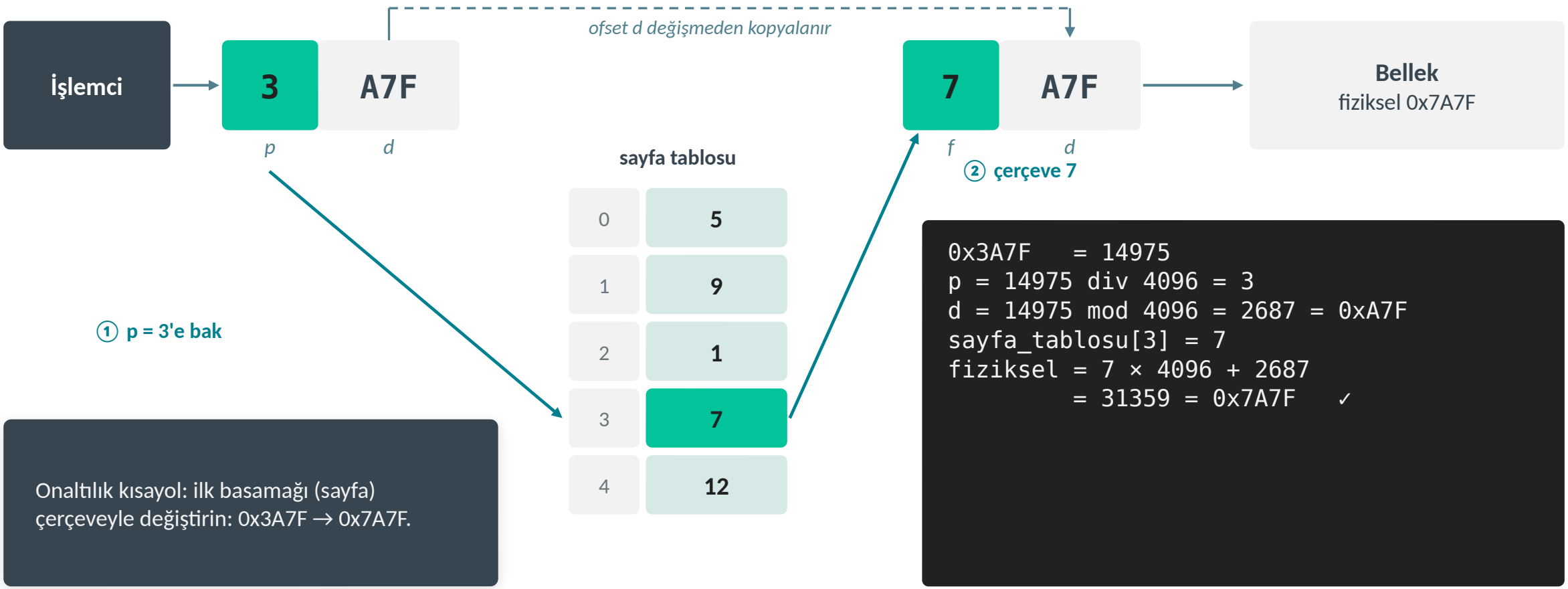
Onaltılık kısayol

- Bir onaltılık basamak = 4 bit; 4 KB sayfada (12 bit) ofset son 3 onaltılık basamaktır.
- 0x3A7F → p = 0x3, d = 0xA7F.
- Ofset değişmeden kopyalanır: yalnızca sayfa numarası çevrilir.

ÇÖZÜMLÜ ÖRNEK

Mantıksal Adres 0x3A7F'nin Çevirisi

16 bitlik adresler, 4 KB sayfalar (n = 12), sayfa tablosu önceki slaytlardaki.

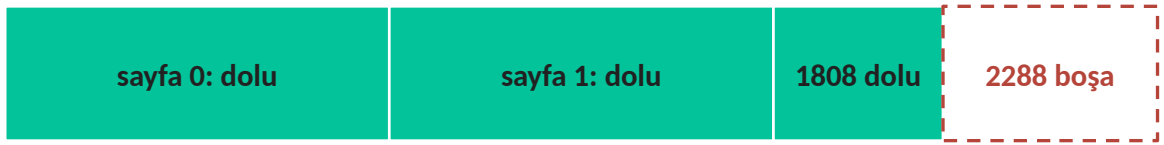


İç Parçalanma ve Sayfa Boyu Seçimi

Sayfalamada dış parçalanma yoktur; ancak bir sürecin son sayfası genellikle dolu değildir.

```
Süreç boyu     10000 bayt
Sayfa boyu      4096 bayt

10000 = 2 × 4096 + 1808
→ 3 çerçeve, sonuncusunda 1808 bayt
→ iç parçalanma = 2288 bayt
```



her 4096 baytta bir çerçeve sınırı

Sayfa boyu bir ödünleşimdir

- **Ortalama israf:** süreç başına yarım sayfa (4 KB sayfada 2 KB); en kötü durumda bir sayfa eksi bir bayt.
- **Küçük sayfalar:** daha az israf, ama daha çok sayfa → daha büyük sayfa tabloları.
- **Büyük sayfalar:** daha küçük tablolar ve daha verimli disk G/Ç'si, ama daha çok israf.
- **x86-64 üzerinde Linux:** normalde 4 KB; veritabanı gibi büyük programlar için 2 MB ya da 1 GB'lık "dev sayfalar" (huge pages).

Sayfa Tablosu Ne Kadar Büyük?

Her sayfa için bir girdi. Büyük adres uzaylarında tablolar çok büyür.

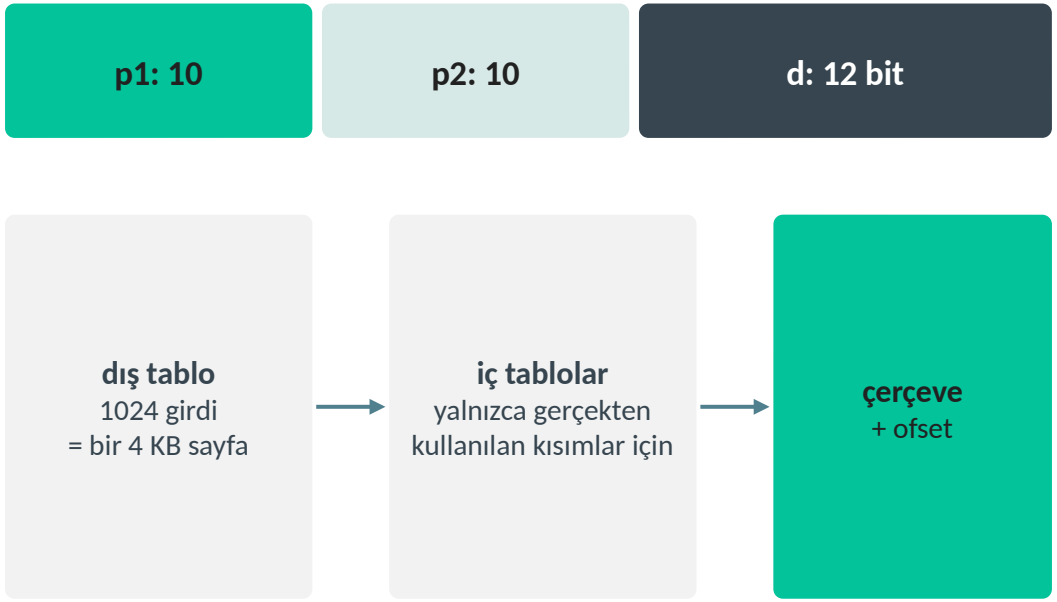
```
32 bit mantıksal adres, 4 KB sayfa, 4 baytlık girdi

sayfa  = 2^32 / 2^12 = 2^20 = 1048576
boyut  = 2^20 × 4 B  = 4 MB   süreç başına!

100 süreç → 400 MB sayfa tablosu
```

Bir sürecin adres uzayının büyük kısmı hiç kullanılmaz; yani bu tablonun çoğu boş girdilerden oluşurdu.

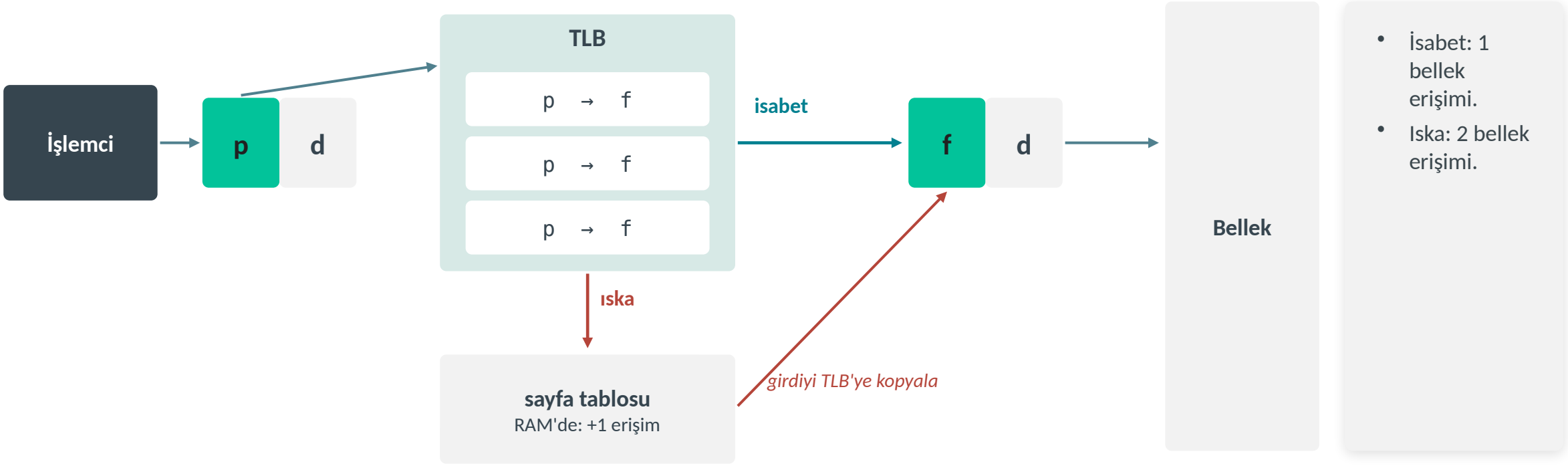
İki düzeyli sayfa tablosu (32 bit)



Sayfa tablosunun kendisi de sayfalanır. x86-64 dört düzey kullanır (9 + 9 + 9 + 9 + 12 = 48 bit); yeni işlemciler beş düzey sunar.

TLB: Sayfa Tablosu Girdileri İçin Bir Önbellek

Sayfa tablosu RAM'de durduğu için her erişim iki bellek erişimi gerektirirdi: biri girdi için, biri veri için.



TLB (translation look-aside buffer, çeviri önbelleği): MMU'nun içinde küçük (≈ 64 – 1536 girdi), çok hızlı, ilişkisel bir bellek.

ÇÖZÜMLÜ ÖRNEK

Etkin Erişim Süresi (EAT)

Bizim uzlaşımımız: TLB arama süresi t_TLB her erişimde sayılır; bir ıska fazladan bir bellek erişimine mal olur.

```
EAT = α (t_TLB + t_m) + (1 − α)(t_TLB + 2 t_m)
    = t_TLB + (2 − α) t_m        α = TLB isabet oranı
```

```
t_TLB = 20 ns,  t_m = 100 ns

α = 0.80:  0.80 × 120 + 0.20 × 220
         = 96 + 44    = 140 ns   (%40 yavaş)
α = 0.98:  0.98 × 120 + 0.02 × 220
         = 117.6 + 4.4 = 122 ns  (%22 yavaş)
TLB yok :  2 × 100    = 200 ns   (2 kat yavaş)
```

Sonucu yorumlamak

- TLB olmasaydı sayfalama her bellek erişimini iki katına çıkarırdı.
- En çok isabet oranı önemlidir: %80 → %98, erişim başına 18 ns kazandırır.
- **Modern sadeleştirme** (Silberschatz 10. baskı): TLB araması başka işlerle örtüştüğü için t_TLB ≈ 0: EAT = α t_m + (1 − α) 2 t_m → α = 0,80 için 120 ns, α = 0,99 için 101 ns.

Koruma Bitleri ve Geçerli–Geçersiz Bit

Her sayfa tablosu girdisi bir çerçeve numarasından fazlasını taşır. MMU bitleri her erişimde denetler.

Sayfa	Çerçeve	Geçerli	İzinler	İçerik
0	5	v	r-x	kod
1	9	v	r-x	kod
2	1	v	rw-	veri
3	7	v	rw-	öbek (heap)
4	12	v	rw-	yığıt (stack)
5 … 15	—	i	—	sürecin parçası değil

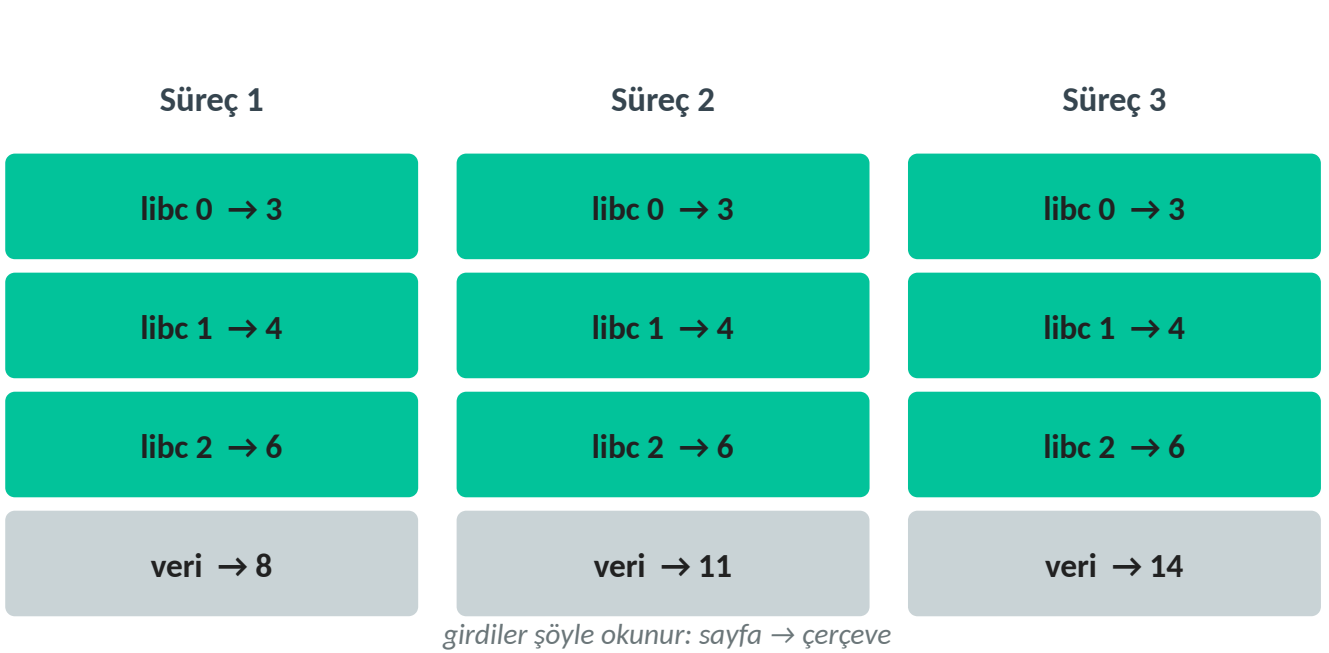
v = geçerli (valid), i = geçersiz (invalid)

Donanım neyi denetler?

- **Geçerli biti = i:** sayfa sürecin adres uzayında değil → tuzak. Mantıksal 0x5000 (sayfa 5) tuzağa düşer.
- **r-x sayfaya yazma:** tuzak. Kodun üzerine yanlışlıkla yazılamaz.
- **rw- sayfayı yürütme:** tuzak (NX biti). Veriye kod enjekte eden birçok saldırıyı durdurur.
- Aynı r, w, x harflerini /proc/self/maps içinde göreceksiniz.

Paylaşılan Sayfalar: Herkes İçin Tek C Kütüphanesi

Salt okunur (yeniden girilebilir, reentrant) kod aynı anda birçok sürece eşlenebilir. Yalnızca veri sayfaları özeldir.





- Üç süreç var, ama libc RAM'de yalnızca bir kez bulunur.
- Paylaşılan sayfalar salt okunur işaretlenir.
- Aynı fikir: fork() başlangıçta tüm sayfaları paylaştırır (yazınca kopyala, 7. hafta).

40 süreç ve 2 MB'lık bir kütüphaneyle paylaşım 39 × 2 MB = 78 MB RAM kazandırır.

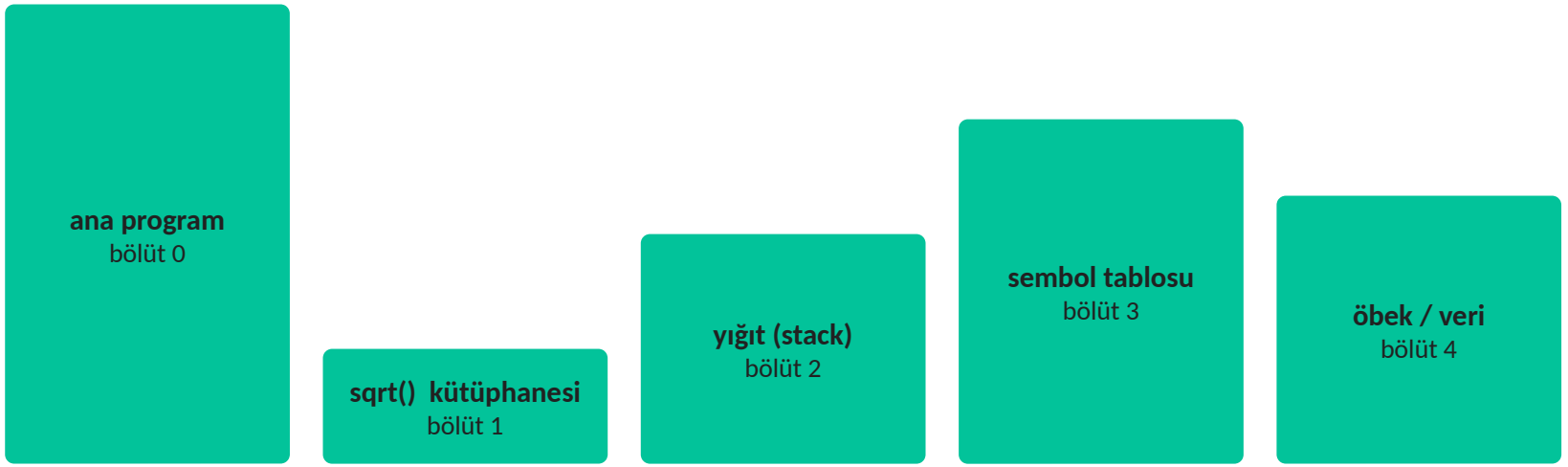
BÖLÜM 3

Bölütleme (Segmentation)

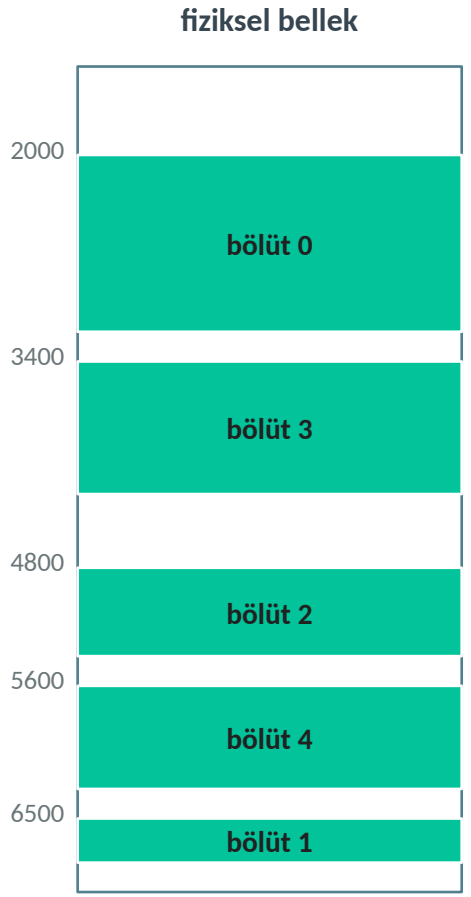
Belleğe programcının gözüyle bakış: farklı boylarda, adı olan parçalardan oluşan bir topluluk.

Programa Kullanıcının Gözüyle Bakış

Programcı 4 KB'lık sayfaları değil; ana programı, fonksiyonları, yığıtı, sembol tablosunu, verileri … düşünür.



mantıksal görünüm: her biri 0 ofsetinden başlayan, farklı boylarda ayrı parçalar



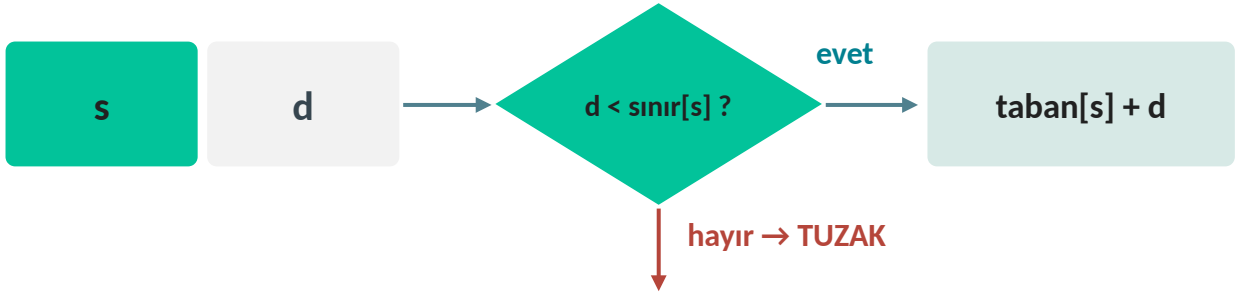
Mantıksal adres = ⟨bölüt numarası s, ofset d⟩. Bölütleri derleyici oluşturur; yükleyici her birine fiziksel bellekte bir taban verir.

Bölüt Tablosu ve Adres Çevirisi

Her bölütün bir tabanı (RAM'de nerede başladığı) ve bir sınırı (uzunluğu) vardır. Ofset sınırla denetlenir.

s	Bölüt	Taban	Sınır
0	ana program	2000	1200
1	sqrt() kütüphanesi	6500	300
2	yığıt	4800	600
3	sembol tablosu	3400	900
4	öbek / veri	5600	700

Taban + sınır fikrinin aynısı; ama her bölüt için bir çift.




```
(s=2, d=53)  : 53 < 600    → 4800 + 53  = 4853
(s=3, d=852) : 852 < 900   → 3400 + 852 = 4252
(s=1, d=299) : 299 < 300   → 6500 + 299 = 6799
(s=0, d=1222): 1222 ≥ 1200 → TUZAK
                (adresleme hatası)
```

KARŞILAŞTIRMA

Sayfalama ve Bölütleme

	Sayfalama	Bölütleme
Birim boyu	sabit (ör. 4 KB), donanım belirler	değişken, programın yapısını izler
Programcı görür mü?	hayır, görünmez	evet: ana program, yığıt, veri …
Mantıksal adres	tek sayı, p | d olarak bölünür	bir çift ⟨s, d⟩
Dış parçalanma	yok	var (değişken boylu bloklar)
İç parçalanma	var, süreç başına ortalama ½ sayfa	yok
Koruma ve paylaşım	sayfa başına	anlamlı birim (bölüt) başına
Tablo girdisi	çerçeve numarası + bitler	taban + sınır + bitler

Birleşim: Multics ve Intel 80386 sayfalardan oluşan bölütler kullandı. Modern 64 bit sistemler yalnızca sayfalamayı tutar.

x86-64 Linux Gerçekte Ne Yapar?

Intel işlemcilerde bölütleme donanımı hâlâ var; ama 64 bit işletim sistemleri onu fiilen devre dışı bırakır.

Bölütleme: düz

- Kod, veri ve yığıt bölütlerinin tabanı 0'dır ve tüm adres uzayını kapsarlar.
- Yalnızca FS/GS kullanımda kalır (iş parçacığına özel veriler için).

Sayfalama: her şey

- 4 düzeyli sayfa tabloları, 48 bit adresler (9+9+9+9+12); yeni işlemcilerde 5 düzey / 57 bit.
- 4 KB, 2 MB, 1 GB'lık sayfalar.

Süreç başına tablolar

- Her sürecin kendi sayfa tabloları vardır; CR3 yazmacı onları gösterir.
- Bağlam değişimi CR3'ü yeniden yükler (3. hafta).

Bölüt fikri yazılımda yaşıyor: Linux her süreç için bir bölge listesi (metin, öbek, yığıt, kütüphaneler) tutar; /proc/self/maps bunu gösterir.

UYGULAMA

Örnek 1: Değişkenlerim Nerede Duruyor?

```
int initialised_global = 42;   /* data segment            */
int uninitialised_global;      /* bss segment (zero-filled) */

int main(void)
{
    int local = 7;                        /* stack */
    int *dynamic = malloc(sizeof(int));   /* heap  */

    printf("text  (code)   main()      : %p\n", (void *)main);
    printf("data  (global) initialised : %p\n",
           (void *)&initialised_global);
    printf("bss   (global) zero-filled : %p\n",
           (void *)&uninitialised_global);
    printf("heap  (malloc)             : %p\n", (void *)dynamic);
    printf("stack (local variable)     : %p\n", (void *)&local);
    printf("page size                  : %ld bytes\n",
           sysconf(_SC_PAGESIZE));
    show_maps();   /* print our lines of /proc/self/maps */
    free(dynamic);
}
/* full program: code/mem_layout.c */
```

Gözlemleyin

- Kod ve global değişkenler: düşük adresler (0x55…).
- Öbek (heap) biraz daha yukarıda; yığıt (stack) en üste yakın (0x7ff…).
- Her adres, /proc/self/maps içinde izinleri uygun bir bölgeye düşer.

```
$ ./mem_layout
text (code) main() : 0x56412f3601e0
data (global) initialised : 0x56412f363010
bss (global) zero-filled : 0x56412f363018
heap (malloc) : 0x564160c6c2a0
stack (local variable) : 0x7ffe905b48c4
page size                  : 4096 bytes
56412f35f000-56412f360000 r--p mem_layout
56412f360000-56412f361000 r-xp mem_layout
56412f361000-56412f362000 r--p mem_layout
56412f362000-56412f363000 r--p mem_layout
56412f363000-56412f364000 rw-p mem_layout
564160c6c000-564160c8d000 rw-p [heap]
7ffe90595000-7ffe905bb000 rw-p [stack]
```

UYGULAMA

Kabuktan Sayfa Boyu ve Bellek Haritası

```
$ getconf PAGESIZE
4096
$ grep Hugepagesize /proc/meminfo
Hugepagesize:       2048 kB
$ head -n 6 /proc/self/maps        # head sürecinin kendi bellek haritası
559948881000-559948883000 r--p 00000000 fe:00 270606         /usr/bin/head
559948883000-559948889000 r-xp 00002000 fe:00 270606         /usr/bin/head
559948889000-55994888b000 r--p 00008000 fe:00 270606         /usr/bin/head
55994888b000-55994888c000 r--p 00009000 fe:00 270606         /usr/bin/head
55994888c000-55994888d000 rw-p 0000a000 fe:00 270606         /usr/bin/head
559970df7000-559970e18000 rw-p 00000000 00:00 0              [heap]
```

Sütunlar

başlangıç–bitiş (sanal), izinler r/w/x + p (özel) ya da s (paylaşılan), dosya ofseti, aygıt, inode, dosya.

Sayfa aritmetiği

0x559948883000 − 0x559948881000 = 0x2000 = 8192 bayt = 2 sayfa. Her sınır 000 ile biter: bölgeler sayfaya hizalıdır.

Aynı dosya, 5 bölge

Tek program dosyası, farklı izinlerle eşlenmiş: başlıklar, kod (r-x), sabitler, veri (rw-).

UYGULAMA

Örnek 2: Aynı Adres, Farklı Değerler

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int x = 100;                    /* one global variable */

int main(void)
{
    pid_t pid = fork();         /* now there are two processes */

    if (pid == 0) {             /* ---- child ---- */
        x = 200;                /* change OUR copy only        */
        printf("child  (PID %d): &x = %p   x = %d\n",
               getpid(), (void *)&x, x);
        return 0;
    }
    wait(NULL);                 /* ---- parent: wait for the child */
    printf("parent (PID %d): &x = %p   x = %d\n",
           getpid(), (void *)&x, x);
    return 0;
}
```

Bu nasıl mümkün?

- fork() ebeveyni kopyalar: aynı düzen, aynı adresler.
- Çocuk x = 200 yapar; ebeveyn hâlâ 100 görür.
- &x mantıksal bir adrestir. Her sürecin kendi sayfa tablosu onu farklı bir çerçeveye eşler.

```
$ ./fork_vaddr
child  (PID 1760): &x = 0x55bc4a29c010  x = 200
parent (PID 1759): &x = 0x55bc4a29c010  x = 100
```

UYGULAMA

Örnek 3: Bit İşlemleriyle Sayfa Numarası ve Ofset

```
#define N_BITS    12                 /* page size 2^12 = 4096 */
#define PAGE_SIZE (1UL << N_BITS)
#define OFF_MASK  (PAGE_SIZE - 1)    /* 0xFFF                 */

/* page -> frame; -1 = invalid page */
int page_table[16] = { 5, 9, 1, 7, 12, -1, /* ... */ };

for (int i = 1; i < argc; i++) {
    unsigned long a = strtoul(argv[i], NULL, 0);
    unsigned long p = a >> N_BITS;      /* high bits  */
    unsigned long d = a & OFF_MASK;     /* low 12 bits */
    if (p >= 16 || page_table[p] < 0) continue;  /* trap */
    unsigned long f = page_table[p];
    unsigned long phys = (f << N_BITS) | d;   /* f*4096 + d */
    printf(... a, p, d, f, phys);
}
/* full program: code/page_split.c */
```

Ne yapıyor?

- p = a >> 12 ve d = a & 0xFFF.
- Çeviri: (frame << 12) | d.
- Slaytlardaki sayfa tablosunun aynısı.

```
$ ./page_split 0x3A7F 0x1234 0x5000
0x3A7F(14975):
  p= 3 d=0xA7F(2687)→frame  7→0x7A7F(31359)
0x1234( 4660):
  p= 1 d=0x234( 564)→frame  9→0x9234(37428)
0x5000(20480):
  p= 5 d=0x000(   0)→invalid: trap to OS
```

KENDİNİZ DENEYİN

Kendi Makinenizde Belleği Keşfedin

```
# Linux (laboratuvar, WSL ya da sanal makine)
$ getconf PAGESIZE          # bayt olarak sayfa boyu
$ cat /proc/self/maps       # cat'in bellek haritası
$ ./mem_layout; ./mem_layout   # iki kez: ASLR
$ free -h                   # kullanılan RAM ve takas
$ cat /proc/swaps           # takas alanları
$ vmstat 1 5                # si/so = takas içeri/dışarı

# Windows
# Görev Yöneticisi > Performans > Bellek:
# "Kullanımda", "Kaydedildi", "Disk belleğine
# alınan havuz"
```

Yanıtlanacak sorular

- Makinenizde sayfa boyu ne kadar? Apple silicon işlemcili bir Mac'te deneyin: 16 KB.
- cat'in [stack] bölgesi kaç sayfa büyüklüğünde?
- mem_layout'un yazdırdığı adresler çalıştırmalar arasında değişiyor mu? Bu güvenlik için neden iyi olabilir?
- Makineniz şu anda takas alanı kullanıyor mu? Ne kadar?

KISA SINAV

Kendinizi Sınayın

1. Taban = 0x2000, sınır = 0x0800. Mantıksal 0x07FF nereye gider? Ya mantıksal 0x0800?

2. Sayfalar 4 KB. Mantıksal adres 0x5C21'i sayfa numarası p ve ofset d olarak bölün.

3. t_TLB = 20 ns, t_m = 100 ns, isabet oranı %90. Etkin erişim süresini hesaplayın.

4. Sayfalamada neden dış parçalanma yoktur, ama iç parçalanma hâlâ vardır?

KAPANIŞ

6. Hafta: Temel Çıkarımlar

- **Mantıksal ve fiziksel:** programlar mantıksal adres kullanır; MMU bunları her erişimde çevirir (yürütme zamanında bağlama).
- **Taban + sınır:** en basit MMU; mantıksal < sınır denetlenir, sonra taban eklenir; aksi hâlde tuzak.
- **Bitişik tahsis:** ilk / en iyi / en kötü uyan; dış parçalanma (%50 kuralı), sıkıştırma pahalıdır.
- **Sayfalama:** sayfa ↔ çerçeve, adres = p | d, süreç başına sayfa tablosu; dış parçalanma yok, yalnızca iç.
- **TLB:** sayfa tablosu girdilerini önbellekte tutar; bizim uzlaşımımızla EAT = t_TLB + (2 − α) t_m.
- **Bölütleme:** bölüt başına taban/sınır ile ⟨bölüt, ofset⟩; x86-64 Linux düz bölütlerle sayfalama kullanır.

GELECEK HAFTA

Sanal Bellek ve Dosya Sistemleri

7. haftanın konuları:

- İsteğe bağlı sayfalama (demand paging) ve sayfa hataları (page fault)
- Sayfa değiştirme: FIFO, OPT, LRU ve boşa çalışma (thrashing)
- Dosya sistemleri: dosya kavramı, erişim yöntemleri, dizin yapıları
- Ara sınavdan önceki son ders (8. hafta: 1–7. haftalar)

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz